FIAP

Bruno Santos Martins
RM: 572073 1CCOB

RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

Santo André - SP
2026

FIAP

Bruno Santos Martins
RM: 572073 1CCOB

RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

Trabalho acadêmico apresentado ao curso de **Ciência da Computação** da **FIAP**, como requisito parcial para avaliação na Atividade Integradora

Santo André - SP
2026

RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

1 Introdução

Este trabalho utiliza como referência a missão **SpaceX CRS-16**, lançada em 5 de dezembro de 2018 por meio do foguete Falcon 9 Block 5, responsável pelo transporte de suprimentos científicos para a Estação Espacial Internacional.

Como os dados completos de telemetria dessa missão não são totalmente públicos, foram utilizados valores simulados baseados em parâmetros típicos de veículos de lançamento orbital. Esses dados representam condições realistas de operação durante a fase inicial de lançamento e foram utilizados para desenvolver a análise de telemetria, o algoritmo de decisão e a simulação computacional apresentados neste estudo.

https://youtu.be/5ZYRqca7OVc?si=ad5cqsisEn2DA50i

2 Organização da Telemetria

A telemetria é o sistema responsável por monitorar e transmitir informações importantes sobre o estado de uma nave ou foguete durante uma missão espacial. Esses dados permitem que os sistemas de controle avaliem se todos os parâmetros estão dentro das condições seguras de operação, garantindo o funcionamento adequado da missão.

Para este trabalho, foi organizada uma tabela de telemetria contendo os principais parâmetros monitorados em um veículo de lançamento. A estrutura da tabela foi dividida em quatro colunas principais: **Parâmetro**, **Valor Usado**, **Limite Seguro** e **Status**. Cada coluna possui uma função específica na análise dos dados coletados.

A coluna **Parâmetro** apresenta as variáveis monitoradas pelo sistema de telemetria, como temperatura, integridade estrutural, energia do sistema e pressão dos tanques. Esses elementos representam informações fundamentais para avaliar a segurança e o desempenho da nave.

A coluna **Valor Usado** representa os valores considerados na simulação do sistema. Esses valores foram definidos com base em parâmetros típicos utilizados em veículos de lançamento orbital.

Já a coluna **Limite Seguro** estabelece os intervalos ou condições consideradas seguras para cada variável monitorada. Caso algum valor ultrapasse esses limites, o sistema pode identificar possíveis falhas ou riscos operacionais.

Por fim, a coluna **Status** indica a condição atual de cada parâmetro. Quando o valor utilizado está dentro dos limites seguros, o sistema apresenta o status **OK**, indicando que o funcionamento está dentro das condições normais de operação.

Os parâmetros analisados incluem:

- **Temperatura Interna**, responsável por monitorar as condições térmicas dos sistemas eletrônicos da nave.

- **Temperatura Externa**, que representa as condições térmicas do ambiente externo durante o voo.

- **Integridade Estrutural**, que indica se a estrutura da nave está estável e sem falhas.

- **Energia do Sistema**, responsável por monitorar a disponibilidade de energia para os sistemas eletrônicos.

- **Pressão dos Tanques**, que acompanha as condições do armazenamento de combustível.

- **Módulos Críticos**, que representam sistemas essenciais da nave, como navegação, comunicação e propulsão.

Todos os parâmetros analisados apresentaram valores dentro dos limites seguros, resultando em um status **OK** para todos os sistemas. Isso indica que, dentro da simulação realizada, a nave estaria em condições adequadas para operar com segurança.

Os dados utilizados nesta telemetria foram baseados em parâmetros típicos de veículos de lançamento orbital, utilizando como referência a missão **CRS-16 do foguete Falcon 9**.

Como os dados completos de telemetria dessa missão não são totalmente públicos, foram utilizados valores simulados dentro de limites operacionais realistas.

Imagem da Telemetria do Foguete Aurora

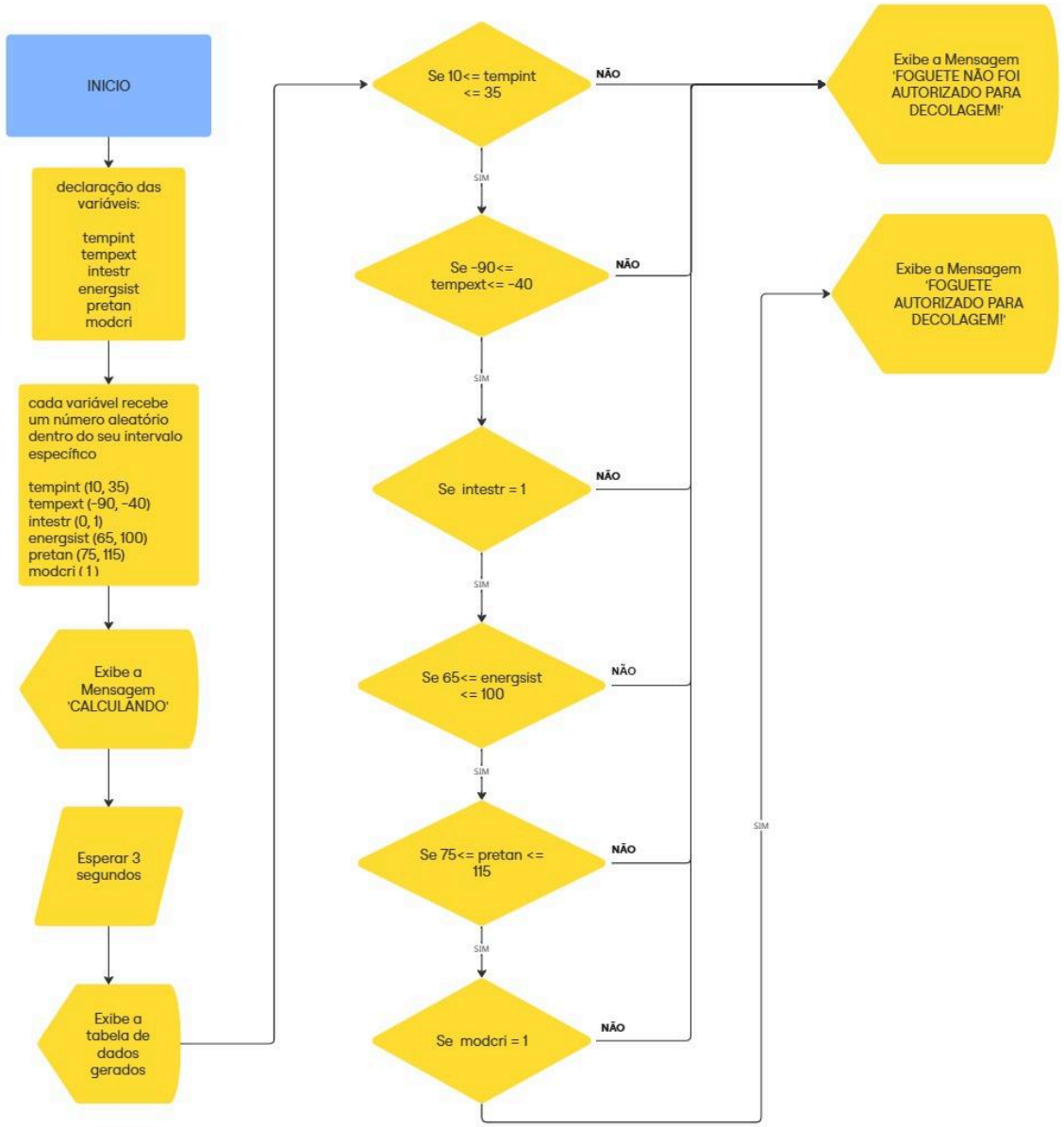
https://github.com/shahar603/Telemetry-Data/tree/master/SpaceX%20CRS-16/JSON

A telemetria representa os dados coletados pelos sensores da nave Aurora Siger antes da decolagem. Esses dados são utilizados para verificar se todos os sistemas estão funcionando dentro das faixas de segurança estabelecidas

3 Algoritmo de Verificação

O fluxograma apresentado representa graficamente o funcionamento do algoritmo desenvolvido em Python para simular um sistema de verificação de segurança antes da decolagem de um foguete. Esse tipo de representação é amplamente utilizado em engenharia e desenvolvimento de software para demonstrar, de forma visual e organizada, a sequência lógica de execução de um programa.

Parâmetro	Valor Usado	Limite Seguro	STATUS
Temperatura Interna	22 °C	15 – 30°C	OK
Temperatura Externa	−65 °C	-80 °C até -50 °C	OK
Integridade Estrutural	1	Deve ser 1	OK
Energia do Sistema	100%	≥ 70%	OK
Pressão dos Tanques	102 bar	85 – 110 bar	OK
Módulos Críticos	1	Deve ser 1	OK



Fluxograma do Código de
Lançamento do Foguete

O fluxo inicia no bloco **“INÍCIO”**, que representa o ponto de partida do algoritmo. A partir desse ponto, o sistema segue para a etapa de **declaração das variáveis**, onde são definidos os parâmetros que serão utilizados durante a execução do programa. Nessa etapa são criadas as variáveis responsáveis por armazenar os valores simulados dos sensores do foguete. As variáveis declaradas são: **tempint**, que representa a temperatura interna do foguete; **tempext**, que representa a temperatura externa do ambiente; **intestr**, que indica a integridade estrutural do foguete; **energsist**, que corresponde ao nível de energia do

sistema; **pretan**, que representa a pressão dos tanques de combustível; e **modcri**, que representa o estado do módulo crítico do sistema.

Após a declaração das variáveis, o fluxograma apresenta a etapa em que **cada variável recebe um valor aleatório dentro de um intervalo previamente definido**. Essa etapa simula o comportamento de sensores reais instalados em sistemas aeroespaciais ou industriais. No fluxograma, são apresentados os intervalos utilizados para cada variável. A temperatura interna (**tempint**) recebe valores entre 10 °C e 35 °C. A temperatura externa (**tempext**) recebe valores entre −90 °C e −40 °C. A integridade estrutural (**intestr**) pode assumir os valores 0 ou 1, representando respectivamente estrutura comprometida ou estrutura segura. O nível de energia do sistema (**energsist**) varia entre 65 e 100. A pressão dos tanques (**pretan**) pode variar entre 75 e 115 bar. Já o módulo crítico (**modcri**) é definido como 1, indicando que o sistema crítico está ativo.

Na sequência, o fluxograma apresenta um bloco de saída onde o sistema **exibe a mensagem "CALCULANDO"**. Esse passo representa o momento em que o sistema está processando os dados obtidos dos sensores. Logo após, ocorre uma etapa de **espera de três segundos**, representando um pequeno atraso no processamento das informações. Essa pausa simula o comportamento de sistemas reais que realizam análises antes de apresentar resultados.

Após a etapa de processamento, o sistema **exibe a tabela com os dados gerados**, mostrando os parâmetros analisados, os valores obtidos e os limites considerados seguros para cada variável. Esse procedimento permite que o usuário visualize claramente todas as informações utilizadas no processo de decisão.

Em seguida, o fluxograma inicia a **etapa de verificação das condições de segurança**, composta por uma sequência de estruturas condicionais representadas por losangos, que indicam pontos de decisão no fluxo do algoritmo.

A primeira verificação analisa se a **temperatura interna do foguete está dentro do intervalo seguro**, representado no fluxograma pela condição "Se 10 ≤ tempint ≤ 35". Caso a condição seja verdadeira, o fluxo segue para a próxima verificação. Caso seja falsa, o sistema segue diretamente para o bloco que exibe a mensagem **"FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM"**.

A segunda verificação analisa a **temperatura externa**, verificando se o valor obtido está dentro do intervalo definido no sistema, representado no fluxograma pela condição "Se −90 ≤ tempext ≤ −40". Se a condição for verdadeira, o fluxo continua para a próxima etapa. Caso contrário, o sistema interrompe a sequência de verificações e exibe a mensagem indicando que a decolagem não foi autorizada.

A terceira etapa verifica a **integridade estrutural do foguete**, representada pela condição "Se intestr = 1". Isso significa que o sistema exige que a estrutura do foguete esteja segura para permitir a continuidade do processo. Se o valor for diferente de 1, o sistema considera que há uma falha estrutural e bloqueia a decolagem.

Na quarta verificação, o sistema analisa o **nível de energia disponível**, representado pela condição “Se 65 ≤ energsist ≤ 100”. Esse parâmetro garante que o foguete possua energia suficiente para operar corretamente durante a fase inicial do lançamento.

Em seguida, ocorre a verificação da **pressão dos tanques de combustível**, representada pela condição “Se 75 ≤ pretan ≤ 115”. Essa etapa é fundamental para garantir que a pressão esteja dentro de níveis seguros, evitando problemas como falta de combustível ou riscos de explosão causados por pressão excessiva.

Por fim, o sistema verifica o **estado do módulo crítico**, representado pela condição “Se modcri = 1”. Esse módulo representa um componente essencial para o funcionamento do sistema. Caso ele não esteja ativo, o lançamento não pode ser autorizado.

A lógica do fluxograma segue o princípio de **validação sequencial**, onde cada condição precisa ser atendida para que o fluxo continue para a próxima verificação. Se qualquer uma das condições falhar, o fluxo é redirecionado imediatamente para o bloco que exibe a mensagem **“FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM”**.

Se todas as verificações forem satisfeitas com sucesso, o fluxo segue até o bloco final que exibe a mensagem **“FOGUETE AUTORIZADO PARA DECOLAGEM”**.

Dessa forma, o fluxograma demonstra de maneira clara e estruturada todo o processo lógico do algoritmo, desde a inicialização do sistema, geração dos dados simulados, processamento das informações, até a tomada de decisão final sobre a autorização ou não da decolagem. Essa representação facilita o entendimento da lógica do programa e permite identificar facilmente a sequência de operações e os critérios utilizados para a tomada de decisão no sistema.

4 Script em Python

Este projeto consiste no desenvolvimento de um programa em Python que simula um sistema de verificação de segurança antes da decolagem de um foguete. Antes que um foguete possa ser lançado, diversos parâmetros precisam ser analisados para garantir que todas as condições estejam dentro de limites seguros de operação. Entre esses parâmetros estão temperatura, pressão, energia do sistema e integridade estrutural do veículo.

O objetivo deste programa é simular esse processo de verificação utilizando valores gerados aleatoriamente para representar sensores do foguete. Após gerar os dados, o programa analisa cada um deles e decide automaticamente se a decolagem será autorizada ou não.

Esse tipo de lógica é semelhante à utilizada em sistemas reais de monitoramento e segurança presentes na engenharia aeroespacial, além de também ser aplicada em sistemas industriais automatizados, onde diversos sensores são analisados continuamente antes da execução de operações críticas.

O código foi desenvolvido utilizando a plataforma **Google Colab**, que permite executar programas em Python diretamente no navegador, sem a necessidade de instalar softwares adicionais no computador.

Código do Programa

A seguir está o código completo utilizado no projeto:

```
#--------------------------
#VERIFICADOR DE DECOLAGEM
#--------------------------

#importação das bibliotecas
import random
from time import sleep

#crição das variáveis
tempint = random.randint(10, 35)
tempext = random.randint(-90, -40)
intestr = random.randint(0, 1)
energsist = random.randint(66, 100)
pretan = random.randint(75, 115)
modcri = 1

print('CALCULANDO DADOS...')
sleep(3)

#criação da tabela dos dados gerados
cabecalho = ['Parâmetro', 'Valor Calculado', 'Limite Seguro']
dados = [
    ['Temperatura Interna', tempint,  '15°C - 30°C' ],
    ['Temepratura Externa', tempext, '-80° até -50°'],
    ['Integridade Estrutural', intestr, 'Deve ser 1'],
    ['Energia do Sistema', energsist, '>= 70'],
    ['Pressão dos Tanques', pretan, '85 - 110 bar'],
    ['Módulo Crítico', modcri, 'Deve ser 1']
]

print(f'{cabecalho[0]:<25} {cabecalho[1]:<25} {cabecalho[2]:25}')
```

```
print('-' *70)

for linha in dados:
  print(f'{linha[0]:30} {linha[1]:<20} {linha[2]:<20}')

#autorização do foguete
if (tempint >= 15 and tempint <= 30) and (tempext >= -80 and tempext <=
50) and (intestr == 1) and (energsist >= 70) and (pretan >= 85 and
pretan <= 110):
  print('=' * 70)
  print('\n')
  print('O FOGUETE FOI AUTORIZADO PARA DECOLAGEM!!')
  print('\n')
  print('=' * 70)
else:
  print('=' * 70)
  print('\n')
  print('O FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM!')
  print('\n')
  print('=' * 70)
```

Importação das Bibliotecas

No início do código são importadas duas bibliotecas importantes da linguagem Python:

import random

from time import sleep

A biblioteca **random** é utilizada para gerar números aleatórios. Esses números representam os valores obtidos por sensores do foguete, como temperatura e pressão.

Já a função **sleep()**, pertencente à biblioteca **time**, é utilizada para criar uma pequena pausa na execução do programa.

No código foi utilizada a instrução:

sleep(3)

Isso faz com que o programa aguarde **3 segundos** antes de continuar sua execução. Essa pausa foi implementada para simular o comportamento de um sistema real analisando dados antes de apresentar o resultado ao operador.

Criação das Variáveis

Após importar as bibliotecas, o programa cria algumas variáveis que representam **parâmetros importantes do foguete**.

```
tempint = random.randint(10, 35)

tempext = random.randint(-90, -40)

intestr = random.randint(0, 1)

energsist = random.randint(66, 100)

pretan = random.randint(75, 115)

modcri = 1
```

Cada uma dessas variáveis representa um sensor ou sistema do foguete:

tempint
Representa a **temperatura interna do foguete**.

tempext
Representa a **temperatura externa do ambiente**.

intestr
Representa a **integridade estrutural do foguete**, podendo ser:

- 0 → Estrutura comprometida
- 1 → Estrutura segura

energsist
Representa o **nível de energia do sistema**.

pretan
Representa a **pressão dos tanques de combustível**.

modcri
Representa o **módulo crítico do sistema**, que precisa estar ativo para permitir a decolagem.

Esses valores são gerados de forma **aleatória**, simulando sensores reais de um sistema.

Simulação de Processamento

O programa exibe a seguinte mensagem:

‘’CALCULANDO DADOS…’’

Depois disso, o programa aguarda **3 segundos** utilizando a função **`sleep(3)`**

Essa pausa foi utilizada para **simular o tempo que um sistema levaria para processar informações antes de apresentar os resultados**.

Criação da Tabela de Dados

Para organizar os dados, o programa cria um **cabeçalho de tabela**:

‘’cabecalho = ['Parâmetro', 'Valor Calculado', 'Limite Seguro']’’

Esse cabeçalho representa três colunas:

-Nome do parâmetro
-Valor gerado pelo sistema
-Limite considerado seguro

Depois disso é criada uma lista chamada **dados**, que contém todas as informações que serão exibidas.

```
‘’dados = [
['Temperatura Interna', tempint, '15°C - 30°C'],
['Temperatura Externa', tempext, '-80° até -50°'],
['Integridade Estrutural', intestr, 'Deve ser 1'],
['Energia do Sistema', energsist, '>= 70'],
['Pressão dos Tanques', pretan, '85 - 110 bar'],
['Módulo Crítico', modcri, 'Deve ser 1']
]’’
```

Essa estrutura organiza os dados em forma de **linhas de tabela**, facilitando a exibição no terminal.

Impressão da Tabela

Para exibir os dados corretamente alinhados no terminal, o programa utiliza **formatação de strings (f-string)**.

Exemplo:

‘’print(f'{cabecalho[0]:<25} {cabecalho[1]:<25} {cabecalho[2]:25}')’’

O comando **<25** significa que o texto ocupará **25 espaços**, garantindo que todas as colunas fiquem alinhadas.

Depois disso, o programa percorre todas as linhas da lista **dados** utilizando um **laço de repetição for**:

‘’for linha in dados:

  print(...)’’

Esse laço permite imprimir automaticamente todos os parâmetros e seus respectivos valores.

Verificação de Segurança

Após exibir a tabela, o programa realiza a verificação de segurança utilizando uma estrutura **if**.

if (tempint >= 15 and tempint <= 30)

and (tempext >= -80 and tempext <= 50)

and (intestr == 1)

and (energsist >= 70)

and (pretan >= 85 and pretan <= 110):

Cada condição representa **um limite de segurança do sistema**.

Por exemplo:

‘’Temperatura interna segura:

15°C até 30°C’’

‘’Pressão dos tanques segura:

85 até 110 bar’’

‘’Integridade estrutural:

Deve ser igual a 1’’

Se **todas as condições forem verdadeiras**, o sistema autoriza a decolagem.

Resultado Final

Se todas as condições forem atendidas, o programa exibe a mensagem:

O FOGUETE FOI AUTORIZADO PARA DECOLAGEM!!

Caso alguma condição não seja atendida, o programa exibe:

O FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM!

Isso garante que o foguete **só decole se todas as verificações estiverem corretas**.

Durante a execução do programa, foram obtidos dois resultados diferentes.

Isso acontece porque os valores são gerados **de forma aleatória pela biblioteca random**.

Primeira execução (Decolagem não autorizada)

Na primeira imagem, alguns parâmetros ficaram **fora do limite seguro**.

```python
#-------------------------
#VERIFICADOR DE DECOLAGEM
#-------------------------

#importação das bibliotecas
import random
from time import sleep

#crição das variáveis
tempint = random.randint(10, 35)
tempext = random.randint(-90, -40)
intestr = random.randint(0, 1)
energsist = random.randint(66, 100)
pretan = random.randint(75, 115)
modcri = 1

print('CALCULANDO DADOS...')
sleep(3)

#criação da tabela dos dados gerados
cabecalho = ['Parâmetro', 'Valor Calculado', 'Limite Seguro']
dados = [
    ['Temperatura Interna', tempint,  '15°C - 30°C' ],
    ['Temepratura Externa', tempext, '-80° até -50°'],
    ['Integridade Estrutural', intestr, 'Deve ser 1'],
    ['Energia do Sistema', energsist, '>= 70'],
    ['Pressão dos Tanques', pretan, '85 - 110 bar'],
    ['Módulo Crítico', modcri, 'Deve ser 1']
]

print(f'{cabecalho[0]:<25} {cabecalho[1]:<25} {cabecalho[2]:25}')
print('-' *70)

for linha in dados:
  print(f'{linha[0]:30} {linha[1]:<20} {linha[2]:<20}')

#autorização do foguete
if (tempint >= 15 and tempint <= 30) and (tempext >= -80 and tempext <= 50) and (intestr == 1) and (energsist >= 70) and (pretan >= 85 and pretan <= 110):
  print('=' * 70)
  print('\n')
  print('O FOGUETE FOI AUTORIZADO PARA DECOLAGEM!!')
  print('\n')
  print('=' * 70)
else:
  print('=' * 70)
  print('\n')
  print('O FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM!')
  print('\n')
  print('=' * 70)
```

```
CALCULANDO DADOS...
Parâmetro                  Valor Calculado           Limite Seguro
----------------------------------------------------------------------
Temperatura Interna            17                    15°C - 30°C
Temepratura Externa            -43                   -80° até -50°
Integridade Estrutural         1                     Deve ser 1
Energia do Sistema             66                    >= 70
Pressão dos Tanques            79                    85 - 110 bar
Módulo Crítico                 1                     Deve ser 1
======================================================================


O FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM!


======================================================================
```

Por exemplo:

Temperatura externa:

-43

Limite seguro:

-80 até -50

Como **-43 está fora desse intervalo**, o sistema considera que a condição externa não é segura para a decolagem.

Por isso o programa exibiu a mensagem:

O FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM!

Segunda execução (Decolagem autorizada)

Na segunda execução, todos os valores gerados ficaram **dentro dos limites permitidos**.

```python
#-------------------------
#VERIFICADOR DE DECOLAGEM
#-------------------------

#importação das bibliotecas
import random
from time import sleep

#crição das variáveis
tempint = random.randint(10, 35)
tempext = random.randint(-90, -40)
intestr = random.randint(0, 1)
energsist = random.randint(66, 100)
pretan = random.randint(75, 115)
modcri = 1

print('CALCULANDO DADOS...')
sleep(3)

#criação da tabela dos dados gerados
cabecalho = ['Parâmetro', 'Valor Calculado', 'Limite Seguro']
dados = [
    ['Temperatura Interna', tempint,  '15°C - 30°C' ],
    ['Temepratura Externa', tempext, '-80° até -50°'],
    ['Integridade Estrutural', intestr, 'Deve ser 1'],
    ['Energia do Sistema', energsist, '>= 70'],
    ['Pressão dos Tanques', pretan, '85 - 110 bar'],
    ['Módulo Crítico', modcri, 'Deve ser 1']
]

print(f'{cabecalho[0]:<25} {cabecalho[1]:<25} {cabecalho[2]:25}')
print('-' *70)

for linha in dados:
  print(f'{linha[0]:30} {linha[1]:<20} {linha[2]:<20}')

#autorização do foguete
if (tempint >= 15 and tempint <= 30) and (tempext >= -80 and tempext <= 50) and (intestr == 1) and (energsist >= 70) and (pretan >= 85 and pretan <= 110):
  print('-' * 70)
  print('\n')
  print('O FOGUETE FOI AUTORIZADO PARA DECOLAGEM!')
  print('\n')
  print('-' * 70)
else:
  print('-' * 70)
  print('\n')
  print('O FOGUETE NÃO FOI AUTORIZADO PARA DECOLAGEM!')
  print('\n')
  print('-' * 70)
```

```
CALCULANDO DADOS...
Parâmetro                 Valor Calculado           Limite Seguro
----------------------------------------------------------------------
Temperatura Interna            24                    15°C - 30°C
Temepratura Externa            -69                   -80° até -50°
Integridade Estrutural         1                     Deve ser 1
Energia do Sistema             75                    >= 70
Pressão dos Tanques            88                    85 - 110 bar
Módulo Crítico                 1                     Deve ser 1
----------------------------------------------------------------------


O FOGUETE FOI AUTORIZADO PARA DECOLAGEM!


----------------------------------------------------------------------
```

Exemplo:

Temperatura interna:

24°C

Limite:

15°C até 30°C

Pressão dos tanques:

88 bar

Limite:

85 até 110 bar

Como **todos os parâmetros estavam dentro das condições de segurança**, o sistema autorizou a decolagem.

Resultado exibido:

O FOGUETE FOI AUTORIZADO PARA DECOLAGEM!

O programa desenvolvido demonstra, de forma simplificada, como funciona um sistema de verificação de segurança antes da decolagem de um foguete. Utilizando sensores simulados e estruturas de decisão da linguagem Python, o sistema consegue analisar diferentes parâmetros operacionais e determinar automaticamente se o lançamento pode ou não ser autorizado.

Essa abordagem reflete conceitos utilizados em sistemas reais de monitoramento, presentes não apenas na engenharia aeroespacial, mas também em diversas áreas da automação industrial e controle de processos.

Além de demonstrar o funcionamento de estruturas condicionais e geração de dados aleatórios, o projeto também mostra como informações podem ser organizadas e exibidas de forma estruturada, facilitando a interpretação dos resultados.

5 Análise Energética do Sistema

A análise energética tem como objetivo verificar se o sistema do foguete possui energia suficiente para realizar o processo de decolagem com segurança. Em sistemas aeroespaciais e em sistemas automatizados de controle, o monitoramento do nível de energia é um fator essencial, pois diversos subsistemas dependem de alimentação elétrica para funcionar corretamente. Entre esses subsistemas estão os computadores de bordo, sensores, sistemas de comunicação, sistemas de navegação e os módulos responsáveis pelo controle e monitoramento da missão.

No algoritmo desenvolvido neste projeto, o nível de energia do sistema é representado pela variável **energsist**, que simula a leitura de um sensor responsável por medir o estado de carga do sistema energético do foguete. Esse valor é gerado de forma aleatória utilizando a biblioteca **random** da linguagem Python, conforme apresentado no código do programa.

A variável é definida da seguinte forma:

energsist = random.randint(65, 100)

Isso significa que o sistema pode gerar valores de energia entre **65% e 100% da capacidade total**, simulando diferentes condições de carga do sistema no momento da verificação de segurança. Essa abordagem permite representar situações reais em que o

nível de energia pode variar dependendo do estado das baterias ou da alimentação elétrica do sistema.

Durante o processo de verificação de segurança, o algoritmo estabelece um limite mínimo de energia necessário para permitir a decolagem. No código, esse limite foi definido como **70% da capacidade energética total do sistema**. Dessa forma, a verificação é realizada por meio da seguinte condição lógica:

energsist ≥ 70

Caso o valor de energia gerado seja inferior a esse limite, o sistema interpreta que não há energia suficiente para garantir o funcionamento adequado dos subsistemas do foguete durante o processo de lançamento. Nesse caso, a decolagem é automaticamente bloqueada pelo sistema de verificação.

Por outro lado, se o nível de energia estiver igual ou superior a 70%, o sistema considera que existe energia suficiente para alimentar os componentes necessários para o início da operação de lançamento, permitindo que o processo continue para as próximas verificações de segurança.

No contexto da simulação desenvolvida, o valor percentual da variável **energsist** representa o nível relativo de energia disponível em relação à capacidade máxima do sistema. Por exemplo, caso o sistema gere um valor de **78%**, isso indica que 78% da capacidade energética total ainda está disponível para uso. Como esse valor está acima do limite mínimo estabelecido, o sistema considera que a condição energética é adequada para a decolagem.

Entretanto, se o valor gerado for **68%**, o sistema identifica que o nível energético está abaixo do limite seguro definido no algoritmo. Nesse cenário, o sistema interpreta que existe risco de falha durante a operação devido à possível falta de energia para alimentar os sistemas críticos, resultando na não autorização da decolagem.

Para complementar essa análise, é possível realizar um cálculo de autonomia energética considerando a capacidade total de armazenamento do sistema. Para fins de simulação, considera-se que o sistema energético do foguete possui uma **capacidade total de 100 kWh** quando totalmente carregado. A partir dessa capacidade, é possível converter o valor percentual de carga em energia real disponível no sistema.

Considerando um exemplo em que o sensor energético indica **78% de carga**, a energia disponível pode ser calculada pela seguinte relação:

Energia disponível = Capacidade total × (Carga atual / 100)

Substituindo os valores:

Energia disponível = 100 × (78 / 100)

Energia disponível = **78 kWh**

Esse valor representa a quantidade de energia efetivamente disponível no sistema antes do processo de decolagem.

Durante a fase inicial de lançamento, diversos sistemas do foguete entram em operação simultaneamente, aumentando o consumo energético. Entre esses sistemas estão os computadores de bordo, sensores, sistemas de navegação, módulos de controle e sistemas de comunicação responsáveis pela telemetria da missão. Para esta análise, considera-se um **consumo estimado de 30 kW de potência durante a fase inicial da decolagem**.

Supondo que essa fase tenha duração aproximada de **5 minutos**, é necessário converter o tempo para horas para realizar o cálculo energético:

5 minutos = **0,083 horas**

A energia consumida durante esse período pode ser calculada pela seguinte expressão:

Energia consumida = Potência × Tempo

Substituindo os valores:

Energia consumida = 30 × 0,083

Energia consumida ≈ **2,49 kWh**

Entretanto, sistemas elétricos reais apresentam perdas energéticas naturais associadas ao funcionamento dos componentes. Essas perdas podem ocorrer devido à dissipação de calor, resistência elétrica dos condutores e eficiência limitada de conversores de energia. Para esta análise, considera-se uma perda energética aproximada de **10% sobre o consumo total do sistema**.

Perdas energéticas = 2,49 × 0,10

Perdas energéticas ≈ **0,25 kWh**

Somando o consumo energético direto com as perdas do sistema, obtém-se o consumo total de energia durante o processo de decolagem:

Energia total utilizada = 2,49 + 0,25

Energia total utilizada ≈ **2,74 kWh**

Com base nesses valores, é possível calcular a autonomia energética restante após a fase inicial de decolagem. Considerando que o sistema possuía inicialmente **78 kWh disponíveis**, e que aproximadamente **2,74 kWh foram utilizados durante o processo de lançamento**, obtém-se:

Energia restante = 78 − 2,74

Energia restante ≈ **75,26 kWh**

A partir dessa análise, observa-se que o sistema possui energia suficiente para realizar a fase inicial de decolagem com segurança, mantendo ainda uma quantidade significativa de energia disponível para as etapas seguintes da missão.

Esse tipo de análise é amplamente utilizado em sistemas aeroespaciais, sistemas embarcados e também em aplicações industriais automatizadas, onde o monitoramento energético é essencial para garantir que operações críticas sejam executadas apenas quando houver recursos energéticos suficientes para manter o funcionamento seguro de todos os subsistemas envolvidos.

6 Análise com Inteligência Artificial

Análise Assistida do Código Python – Sistema de Verificação de Decolagem

O código desenvolvido em Python tem como objetivo simular um sistema de verificação de segurança utilizado antes da decolagem de um foguete. Em operações aeroespaciais reais, diversos parâmetros precisam ser analisados para garantir que todas as condições operacionais estejam dentro de limites seguros antes da autorização de lançamento. Dessa forma, o algoritmo criado utiliza valores aleatórios para representar sensores do foguete e verificar automaticamente se as condições simuladas permitem ou não a decolagem.

Inicialmente, o programa realiza a importação das bibliotecas necessárias. A biblioteca **random** é utilizada para gerar valores aleatórios que simulam dados de sensores do foguete, enquanto a biblioteca **time** é utilizada para inserir uma pequena pausa no programa, simulando o tempo de processamento de um sistema real de análise de dados. Em seguida, são definidas variáveis que representam parâmetros importantes para a segurança da missão, como temperatura interna, temperatura externa, integridade estrutural, energia do sistema, pressão dos tanques e estado de um módulo crítico.

Esses dados podem ser classificados em três categorias principais. A primeira categoria corresponde aos **dados ambientais**, que incluem a temperatura interna e a temperatura externa. Esses parâmetros são importantes para garantir que os equipamentos eletrônicos e os sistemas do foguete operem dentro de faixas térmicas adequadas. A segunda categoria corresponde aos **dados estruturais**, que incluem a integridade estrutural do foguete e o estado do módulo crítico. Esses parâmetros estão diretamente relacionados à segurança física da estrutura do veículo espacial. A terceira categoria corresponde aos **dados operacionais**, como energia do sistema e pressão dos tanques de combustível, que garantem que o foguete possua condições técnicas adequadas para iniciar o processo de lançamento.

Após a geração dos valores simulados, o programa organiza os dados em uma estrutura semelhante a uma tabela, apresentando os parâmetros analisados, os valores calculados e

os limites considerados seguros para cada variável. Essa apresentação facilita a visualização e a interpretação dos dados, simulando um painel de monitoramento utilizado em centros de controle de lançamento.

Durante a análise do código, foram identificadas algumas possíveis anomalias relacionadas à lógica do algoritmo. A primeira anomalia está relacionada à verificação da temperatura externa. Embora a tabela indique que o intervalo seguro está entre -80 °C e -50 °C, a condição utilizada no algoritmo permite valores até 50 °C, o que não corresponde ao limite estabelecido. Essa inconsistência pode permitir que o sistema autorize uma decolagem em condições térmicas inadequadas. Outra anomalia observada é que o parâmetro referente ao módulo crítico foi definido no programa, porém não foi incluído na condição lógica responsável por autorizar ou bloquear a decolagem. Em um sistema real, a ausência dessa verificação poderia permitir que o foguete fosse liberado para lançamento mesmo com falha em um componente crítico.

Também foi observado que alguns intervalos de geração de dados são mais amplos que os limites seguros definidos. Por exemplo, a temperatura interna pode variar de 10 °C a 35 °C, enquanto o intervalo seguro está entre 15 °C e 30 °C. Esse comportamento pode ser interpretado como uma simulação de possíveis falhas de sensores ou de condições ambientais adversas, o que é comum em testes de sistemas de monitoramento.

Além das anomalias lógicas, também foi identificado um pequeno erro de digitação na tabela exibida pelo programa, onde a palavra “Temperatura” foi escrita incorretamente. Embora esse erro não afete o funcionamento do algoritmo, ele pode comprometer a clareza da apresentação dos dados em um relatório técnico.

Do ponto de vista da análise de risco, alguns cenários críticos podem ser identificados. Caso a integridade estrutural do foguete esteja comprometida e o sistema não detecte corretamente essa falha, existe o risco de ruptura estrutural durante a decolagem. Da mesma forma, pressões fora do intervalo seguro nos tanques de combustível podem gerar situações perigosas, como falta de combustível ou até mesmo explosões causadas por sobrepressão. Outro fator importante é a energia do sistema: valores abaixo do limite mínimo podem causar falhas nos computadores de bordo, sistemas de navegação ou comunicação. Por fim, temperaturas internas elevadas podem levar ao superaquecimento de componentes eletrônicos, comprometendo o funcionamento dos sistemas de controle.

De forma geral, o algoritmo apresenta uma estrutura lógica clara e organizada, sendo capaz de simular um sistema básico de verificação de segurança para lançamento de foguetes. O uso de condições booleanas para validação dos parâmetros, juntamente com a geração de dados aleatórios, permite criar diferentes cenários de teste para o sistema. Entretanto, algumas melhorias podem tornar o programa mais robusto e mais próximo de aplicações reais. Entre essas melhorias, destacam-se a correção dos limites de verificação de temperatura externa, a inclusão da verificação do módulo crítico na condição de autorização de decolagem e a implementação de mensagens de erro específicas que indiquem exatamente qual parâmetro está fora do limite seguro.

Conclui-se que o código representa uma simulação funcional de um sistema de verificação pré-lançamento, permitindo compreender de forma prática como dados de sensores podem ser analisados automaticamente para apoiar decisões críticas em operações aeroespaciais.

Com pequenas melhorias estruturais e ajustes na lógica de validação, o programa pode evoluir para um modelo mais completo de monitoramento e tomada de decisão em sistemas de segurança de lançamento.

7 Reflexão Crítica

8 Conclusão